SESSION 3

Develop AI Agents with AI

Design, build, and rehearse practical AI agents with tools, memory, and retrieval

90 min | Advanced

What Is an AI Agent?

> *An AI agent is an LLM loop that can plan steps, call tools, store useful state, and continue until a goal is complete.*

Aspect	Sub-Agent (S2)	Full Agent (S3)
Scope	Single workflow step	End-to-end goal
Tools	Read/write files	CLI, APIs, search, code exec
Memory	Session context	Persistent across runs
Autonomy	Human triggers each step	Plans and executes independently

The Agent Loop

Observe
Read context

→

Plan
Choose next action

→

Act
Execute with tools

→

Evaluate
Check result

→

Store
Save learnings

Design principle: Every loop should be auditable. If you cannot explain why the agent took an action, the agent is not production-ready.

Core Agent Capabilities

Planning

Breaks goals into next actions

Tool Use

Runs commands, APIs, structured ops

Memory

Remembers context, outcomes, patterns

Delegation

Hands off subtasks to specialists

Skills (S1) → **Sub-Agents** (S2) → **Full Agents** (S3) → **Factories** (S4)

MCP vs CLI Tools

CLI

PROS

- ✓ 30%+ better token efficiency
- ✓ Unix-style composability
- ✓ Human-readable debugging
- ✓ Mature ecosystem

CONS

- ✗ Can hallucinate syntax
- ✗ Harder for stateful tasks
- ✗ Platform-dependent

MCP

PROS

- ✓ Type-safe JSON schemas
- ✓ Complex integrations
- ✓ Human-in-loop approvals
- ✓ Growing ecosystem

CONS

- ✗ Context bloat
- ✗ Operational complexity
- ✗ Often worse in benchmarks

CLI outperforms for existing tools. **MCP** for specialized/secure workflows.

EXERCISE 1: Invent a New Tool

The Prompt

```
Invent a new tool: [your tool description]
```

The invent workflow in AGENTS.md handles the rest:

1 Implement

AI writes the CLI tool based on your description

2 Test

AI generates and runs tests to verify it works

3 Document

AI creates usage docs and registers the tool

Tool ideas: Weather API CLI • GitHub PR summarizer • Slack notifier • Log analyzer • DB health check • anything you want

You describe it. The AI builds, tests, and documents it — 10 min

Memory Patterns for Agents

Working Memory

Current task / session

Keep immediate plan and observations

Episodic Memory

Past runs

Reuse successful strategies, avoid repeated mistakes

Knowledge Memory

Indexed docs / data

Answer with grounded evidence (RAG)

Keep short-term lightweight • Persist only high-value learnings • Tag memory with source + timestamp

RAG for Agents

Documents → Chunk → Embed → Store (ChromaDB) → Retrieve → Augment Prompt → Grounded Answer

Approach	Characteristics
Plain LLM	May hallucinate, uses training data only
RAG Query	Grounded in indexed content
Agent + RAG	Decides when to retrieve, combines with tools

> What changes: retrieval becomes a tool call in the plan. Memory and retrieval complement each other. Answers become traceable, not just plausible.

EXERCISE 2: RAG-Backed Memory Skill

The Prompt

```
Create a RAG-backed memory skill: [your description]
```

AI generates: embed + store to vector DB on trigger, semantic retrieve on recall

Skill ideas (all RAG-backed):

`remember-learnings`	Embed lessons learned → retrieve similar past insights when facing new problems
`project-context`	Index project docs (stack, conventions) → semantic search for grounded answers
`decision-log`	Embed architectural decisions + rationale → query "why did we choose X?"
`error-patterns`	Embed bug reports + fixes → auto-retrieve matching solutions for new errors

Memory + RAG: you describe the pattern, AI builds the embed/store/retrieve skill — 10 min

Plan Your Agent

Work in teams — design an agent for a real workflow

1 Pick a Use Case

Choose a real workflow from your team's daily work. Example: PR review, deploy pipeline, incident triage

2 Define Tools

Which CLI/MCP tools?
Which ones from Exercise 1?
What new tools are needed?
What APIs to integrate?

3 Design Memory

RAG-backed memory from Exercise 2. What to embed?
What retrieval triggers?
What to keep ephemeral?

Guardrails: blocked patterns, stop conditions, approval gates

Subagents: which tasks to delegate to specialist agents?

EXERCISE 3: Build the Agent

Agent Architecture

UI Layer	Chat interface, tool output display
Agent Loop	Observe → Plan → Act → Evaluate
Tool Registry	Your tools from Exercise 1 + built-ins
RAG Memory	Your skill from Exercise 2
Subagents	Delegate: review, test, doc generation
Guardrails	Blocked patterns, approval gates

Build checklist:

UI

Chat UI that shows
agent thinking + tool calls

Tools + Memory

Integrate tools from Ex 1
+ RAG memory from Ex 2

Safety

Add guardrails:
blocked patterns, approvals

Delegation

Generate subagents:
review-agent, test-agent, etc.

Teams build from gemini_agent.py — combine tools + memory + subagents + guardrails — 15 min

Session 3 Summary

1. Understood agent foundations: loop, planning, capabilities
2. Chose CLI vs MCP with clear tradeoffs (pros/cons)
3. Invented a custom tool using the AGENTS.md workflow
4. Created a RAG-backed memory skill (embed + store + retrieve)
5. Planned and built a full agent as a team (tools + memory + guardrails)

> **AI agents are not "just prompts." They are engineered loops with planning, tools, memory, and verification.**